2026 ACM Programming Contest

- Complete as many questions as you can during the contest.
- **Only one computer may be used per team.**
- No connection to the Internet may be made during the contest.
- All questions require you to read the test data from standard input and write results to standard output; you cannot use any non-contest files for input or output.
- The input to problems may consist of one or more test cases, read the problem statement carefully.
- Programming style is not considered in this contest. You are free to code in whatever style you prefer. Commenting code is not required.
- All communication with the judges will be handled by the PC2 environment.
- Allowed programming languages: C, C++, Java, and Python.
- **Java users: remember to call `nextLine()` on a scanner after reading in an integer when needed.**
- All programs will be re-compiled prior to testing with the judges' data.
- Output must match the sample format exactly. Do not print prompts unless asked to.
- Non-standard libraries cannot be used in your solutions. The Standard Template Library (STL) and string libraries are allowed. The standard Java API is available and can be used, except for those packages that are deemed dangerous by contest officials (e.g., that might generate a security violation)
- Hardcopy reference materials are allowed, but no electronic material. Calculators are fine, but no laptops, phones, USB drives, etc. are allowed.
- **No use of the internet or generative AI is permitted during the competition!!**

Judges decisions are to be considered final. No cheating will be tolerated.

Problem A - Don't Make Them Cry

In response to feedback that her piracy courses are too difficult, respected pirate Captain Makum-Crie is evaluating new grading policies. She believes that if at least 20% of pirates in each of her courses pass with a grade of D or higher, the course has the right difficulty level. One new policy being considered is to exclude the lowest N grades each pirate earns from that pirates's final grade calculation in a course. She needs a program that determines the value of N necessary for 20% of her pirates to pass the course.

Input The input file starts with a line containing a single positive integer number of courses taught by Captain Makum-Crie. The remainder of the input file contains grades for each assignment for each course. Data for each course starts with a line containing a single course name e.g. "PS101" without quotation marks (Pirate-Science 101). The next line after the course name contains a single integer, A ($0 < A < 32$), which identifies the number of assignments in the course. The number of assignments is followed by a single line containing the number of pirate, S ($0 < S < 100$). The next line is a list of PirateId Grade pairs per assignment. PirateIds are integers I ($0 < I < 16777216$). Grades are floating point numbers G ($0.0 \leq G \leq 1.0$).

An average grade of 0.6 is the lowest passing grade.

Output The program output is the course name followed by one space and then the integer number of lowest grades to drop so that at least 20% of pirates pass the course or the string `Course is difficult.` without quotes if there is no number of dropped grades that will result in 20% or more pirates passing the course.

Sample Input

```
3
PS101
3
4
9 0.5 3 0.56 51 0.05 1 0.0
3 0.0 51 0.0 9 0.0 1 0.0
51 0.0 3 0.0 9 0.4 1 0.5
CS310
3
4
9 0.9 3 0.0 51 0.05 1 0.8
```

```
3 0.0 51 0.5 9 0.0 1 0.0
51 0.0 3 0.39 9 0.0 1 0.2
CS200
3
3
9 0.6 3 0.6 51 0.05
3 1.0 51 0.6 9 0.0
51 0.0 3 0.0 9 0.0
```

Sample Output

```
PS101 Course is difficult.
CS310 2
CS200 1
```

Problem B - Drop Two

Captain Henry Avery is trying to trim the fat from his crew to find their true strength. He has assigned performance scores for each crew member (some positive, some negative.) The calculate the crews average score, he decides to have the two crew members whose scores have the smallest overall magnitude walk the plank.

Given lists of numbers, calculate the average of the numbers in each list when the two lowest magnitude (regardless of sign) numbers in each list are omitted from the average calculation.

Input The input file starts with a line containing a single integer number of lists followed by one line of space delimited integers per list. There will be at least 3 numbers in every list.

Output The output consists of the calculated average of each list on separate lines.

Sample Input

```
4
10 5 1 30 15 5
-5 1 -1 5 6 -6 7
3 3 3
51 51 51 0 0 51
```

Sample Output

```
15
1
3
51
```

Problem C - Fire the Cannons!

Captain Bytebeard has finally caught up with a rival pirate ship, but the enemy is attempting to escape. Along one side of Captain Bytebeard's ship is a row of cannons, each with a different maximum firing distance.

Captain Bytebeard's crew considers each cannon in order from left to right. A cannon can hit the enemy ship if its maximum firing distance is greater than or equal to the current distance to the enemy ship. However, firing a cannon creates enough recoil that the cannon directly next to it cannot safely be fired. If a canon is incapacitated by a nearby cannon firing, it cannot fire again for the remainder of the current battle. Each cannon can only fire at most one time. Cannons only fire one at a time.

The enemy ship is also moving away. Each time a cannon is successfully fired, the enemy ship moves 10 units farther away before the crew considers the next cannon.

Captain Bytebeard needs a program to determine how many cannons will successfully fire and hit the enemy ship before it escapes out of range.

Input The input file starts with a line containing a single positive integer representing the number of battles.

Data for each battle starts with a line containing a single ship name. Ship names contain no spaces.

The next line contains a single integer, C ($0 < C < 100$), representing the number of cannons on the ship.

The next line contains a single integer, D ($0 < D < 10000$), representing the starting distance to the enemy ship.

The next line contains C integers. Each integer R ($0 < R < 10000$) represents the maximum firing distance of a cannon. Cannons are listed in their physical order from left to right along the side of the ship.

A cannon successfully fires if its maximum firing distance is greater than or equal to the current distance to the enemy ship and the cannon directly before it did not fire.

After each successful shot, the enemy ship moves 10 units farther away.

Output For each battle, output the ship name followed by one space and then the number of cannons that successfully fire and hit the enemy ship.

If no cannon can successfully hit the enemy ship, output `0`.

Sample Input

```
3
BlackPearl
5
50
60 70 20 80 90
JollyRoger
5
50
60 60 60 60 60
FlyingDutchman
4
75
20 30 40 50
```

Sample Output

```
BlackPearl 2
JollyRoger 2
FlyingDutchman 0
```

Problem E - Grid Coloring

In an attempt to map enemy territory, Captain Bytebeard is trying to color his map accordingly. Blue for friendly pirates, Red for enemy pirates.

You have an m-by-n grid of squares that you wish to color. You may color each square either red or blue, subject to the following constraints:

- Every square must be colored.
- Colors of some squares are already decided (red or blue), and cannot be changed.
- For each blue square, all squares in the rectangle from the top left of the grid to that square must also be blue.

Given these constraints, how many distinct colorings of the grid are there? The grid cannot be rotated.

Input The first line of input consists of two space-separated integers m and n ($1 \leq m, n \leq 30$).

Each of the next m lines contains n characters, representing the grid. Character `B` indicates squares that are already colored blue. Similarly, `R` indicates red squares. Character `.` indicates squares that are not colored yet.

Output Print, on a single line, the number of distinct colorings possible.

For the first sample, the 6 ways are:

```
BB  BB  BR  BR  BB  BB
BB  BR  BR  BR  BR  BB
BR  BR  BR  RR  RR  RR
```

Sample Input 1

```
3 2
..
B.
.R
```

Sample output 1

```
6
```

Sample Input 2

```
7 6
......
.....B
.B..R.
......
...B..
.R....
...R..
```

Sample Output 2

```
3
```

Problem E - Easier Done than Said?

Captain Two-Toes has decided to begin locking all of his treasure in a password-protected vault on his ship. Two-Toes wants to ensure that only he is able to access his treasure, and none of his pirate constituents are able to steal his booty.

Password security is a tricky thing. Pirates prefer simple passwords that are easy to remember (like buddy), but such passwords are often insecure. Some captains use random computer-generated passwords (like xvtpzyo), but most pirates have a hard time remembering them and sometimes leave them written on a note in their quarters. One potential solution is to generate "pronounceable" passwords that are relatively secure but still easy to remember.

Two-Toes is developing such a password generator. You work as the Quartermaster, and it's your job to test the generator and make sure that the passwords are acceptable. To be acceptable, a password must satisfy these three rules: 1. It must contain at least one vowel. 2. It cannot contain three consecutive vowels or three consecutive consonants. 3. It cannot contain two consecutive occurrences of the same letter, except for 'ee' or 'oo'.

(For the purposes of this problem, the vowels are 'a', 'e', 'i', 'o', and 'u'; all other letters are consonants.) Note that these rules are not perfect; there are many common/ pronounceable words that are not acceptable.

Input The input consists of one or more potential passwords, one per line, followed by a line containing only the word `end` that signals the end of the file. Each password is at least one and at most twenty letters long and consists only of lowercase letters.

Output For each password, output whether or not it is acceptable, using the precise format shown in the example.

Sample Input

```
a
tv
ptoui
bontres
zoggax
wiinq
eep
houctuh
end
```

Sample output

```
<a> is acceptable.
<tv> is not acceptable.
<ptoui> is not acceptable.
<bontres> is not acceptable.
<zoggax> is not acceptable.
<wiinq> is not acceptable.
<eep> is acceptable.
<houctuh> is acceptable.
```

Problem F - Star Arrangements

The recent vote in Libertalia favoring United Pirates States statehood has made flag makers very excited. An updated flag with 51 stars rather than the current one with 50 would cause a huge jump in U.P.S. flag sales. The current pattern for 50 stars is five rows of 6 stars, interlaced with four offset rows of 5 stars. The rows alternate until all stars are represented.

```
* * * * * *
 * * * * *
* * * * * *
 * * * * *
* * * * * *
 * * * * *
* * * * * *
 * * * * *
* * * * * *
```

This pattern has the property that adjacent rows differ by no more than one star. We represent this star arrangement compactly by the number of stars in the first two rows: $6, 5$.

A 51-star flag that has the same property can have three rows of 9 stars, interlaced with three rows of 8 stars (with a compact representation of $9, 8$). Conversely, if a state were to leave the United Pirate States, one appealing representation would be seven rows of seven stars $(7, 7)$.

A flag pattern is *visually appealing* if it satisfies the following conditions: - Every other row has the same number of stars. - Adjacent rows differ by no more than one star. - The first row cannot have fewer stars than the second row.

Your team sees beyond the short-term change to 51 for the U.S. flag. You want to corner the market on flags for any union of three or more states. Given the number S of stars to draw on a flag, find all possible *visually appealing* flag patterns.

Input The input consists of a single line containing the integer S ($3 \leq S \leq 32,767$).

Output On the first line, print S, followed by a colon. Then, for each visually appealing flag of S stars, print its compact representation, one per line.

This list of compact representations should be printed in increasing order of the number of stars in the first row; if there are ties, print them in order of the number of stars in the second row. The cases 1-by-S and S-by-1 are trivial, so do not print those arrangements.

The compact representations must be printed in the form "x,y", with exactly one comma between x and y and no other characters.

Sample Input

```
50
```

Sample Output

```
50:
2,1
2,2
3,2
5,4
5,5
6,5
10,10
13,12
17,16
25,25
```

Problem G - Coconuts

The short story titled Coconuts, by Captain Williams, appeared in the Pirate Evening Post on October 9, 1926. The story tells about five pirates and a monkey who were shipwrecked on an island. They spent the first night gathering coconuts. During the night, one pirate woke up and decided to take his share of the coconuts. He divided them into five piles. One coconut was left over so he gave it to the monkey, then hid his share and went back to sleep.

Soon a second pirate woke up and did the same thing. After dividing the coconuts into five piles, one coconut was left over which he gave to the monkey. He then hid his share and went back to bed. The third, fourth, and fifth man followed exactly the same procedure. The next morning, after they all woke up, they divided the remaining coconuts into five equal shares. This time no coconuts were left over.

An obvious question is "how many coconuts did they originally gather?" There are an infinite number of answers, but the lowest of these is 3,121. But that's not our problem here.

Suppose we turn the problem around. If we know the number of coconuts that were gathered, what is the maximum number of persons (and one monkey) that could have been shipwrecked if the same procedure could occur?

Input The input will consist of a sequence of integers, each representing the number of coconuts gathered by a group of persons (and a monkey) that were shipwrecked. The sequence will be followed by a negative number.

Output For each number of coconuts, determine the largest number of persons who could have participated in the procedure described above. There may be no solution for some of the input cases; if so, state that observation.

Sample Input

```
25
30
3121
-1
```

Sample Output

```
25 coconuts, 3 persons and 1 monkey
30 coconuts, no solution
3121 coconuts, 5 persons and 1 monkey
```

Problem H - Secret Chamber

By now you have probably heard that there is a spectacular stone sculpture featuring four famous pirates at Mount Rushmore. However, very few people know that this monument contains a secret chamber. This sounds like something out of a plot of a Hollywood movie, but the chamber really exists. It can be found behind the head of Pirate Lincoln and was designed to serve as a Hall of Records to store important historical pirate documents and treasure. Historians claim that the construction of the hall was halted in 1939 and the uncompleted chamber was left untouched until the late 1990s, but this is not the whole truth.

In 1982, the famous archaeologist Davida Jones secretly visited the monument and found that the chamber actually was completed, but it was kept confidential. This seemed suspicious and after some poking around, she found a hidden vault and some documents inside. Unfortunately, these documents did not make any sense and were all gibberish. She suspected that they had been written in a code, but she could not decipher them despite all her efforts.

Earlier this week when she was in the area to follow the trail of famous pirate Henry Avery, Dr. Jones finally discovered the key to deciphering the documents.

She found a document that contains a list of translations of letters. Some letters may have more than one translation, and others may have no translation. By repeatedly applying some of these translations to individual letters in the gibberish documents, she might be able to decipher them to yield historical pirate documents such as the Declaration of Pirate and the Document of Constitutional Piracy. She needs your help.

You are given the possible translations of letters and a list of pairs of original and deciphered words. Your task is to verify whether the words in each pair match. Two words match if they have the same length and if each letter of the first word can be turned into the corresponding letter of the second word by using the available translations zero or more times.

Input The first line of input contains two integers m ($1 <= m <= 500$) and n ($1 <= n <= 50$), where m is the number of translations of letters and n is the number of word pairs. Each of the next m lines contains two distinct space-separated letters a and b, indicating that the letter a can be translated to the letter b. Each ordered pair of letters (a, b) appears at most once. Following this are n lines, each containing a word pair to check. Translations and words use only lowercase letters 'a'–'z', and each word contains at least 1 and at most 50 letters.

Output For each pair of words, display *yes* if the two words match, and *no* otherwise.

Sample Input

```
9 5
c t
i r
k p
o c
r o
t e
t f
u h
w p
we we
can the
work people
it of
out the
```

Sample Output

```
yes
no
no
yes
yes
```

Problem I - Tarot Sham Boast

Curse your rival, Captain Blackbear! Every year at the annual Pirate Rock Paper Scissors tournament, you have made it to the final match. (Your Rock technique is unmatched, and your Paper cuts to the bone! Your Scissors need a little work, though.) But every year, Blackbeard defeats you, even though his moves appear entirely random! And he claims to the press that he simply cannot be beaten. What is his secret?

Fortunately, you think you have figured it out. This year, just before the tournament, you caught him visiting various shamans around town. Aha! He is using the supernatural against you! You figured two can play at this game. So you went and visited a set of fortune-tellers, who have each used a Tarot deck to predict a sequence that your rival will end up using, sometime during the match.

However, your initial excitement has passed, and now you are feeling a little silly. This cannot possibly work, right? In the end it feels like you have paid good money for a fraudulent, random set of predictions. Oh well; you might as well keep an eye out for some of them during the match. But which predictions will you use?

In the final match, you and your rival will play n rounds of Rock Paper Scissors. In each round, your rival and you will both choose one of the three options (Rock, Paper, or Scissors). Based on your selections, a winner of the round will be determined (exactly how is irrelevant to this problem).

Given the length of the final match and the various predictions, sort them in order of how likely they are to appear sometime during the match as a contiguous sequence of options chosen by your rival, assuming he is choosing his symbol in each round independently and uniformly at random.

Input The first line of input contains two integers n ($1 <= n <= 10^6$), the number of rounds in the final match, and s ($1 <= s <= 10$), the number of sequences. The remaining s lines each describe a prediction, consisting of a string of characters 'R', 'P', and 'S'. All predictions have the same length, which is between 1 and n characters long, inclusive, and no longer than 10^5.

Output Display all of the predictions, sorted by decreasing likelihood of appearance sometime during the final match. In the case of tied predictions, display them in the same order as in the input.

Sample Input

```
20 3
PRSPS
SSSSS
PPSPP
```

Sample Output

```
PRSPS
PPSPP
SSSSS
```

Problem J - Hash Map

Captain Sha Two-Fifty-Six buried her gold on an island laid out like a tic-tac-toe board:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
```

Her maps look like gibberish: a starting square followed by a list of words. The secret is to take the SHA-256 hash of each word and read the first hexadecimal character of the hash. That character is the square you step to next.

A map is genuine only if every step is legal:

The hash's first character must be a digit from `1` to `9`. If it's `0` or `a-f`, you've walked off the island.

The new square must be adjacent to your current square: directly up, down, left or right. Diagonal moves and staying on the same square aren't allowed.

If every word in a genuine map passes, the treasure is buried on the last square you reach.

Hint: Being a big security buff, captain Sha is a huge fan of the following libraries:

- python hashlib
- java.security.MessageDigest (note, outputs a byte array, not characters!)

Input The first line has T, the number of maps. Each of the next T lines has a starting square $(1-9)$ followed by one or more lowercase words, separated by spaces. Hash each word as UTF-8 bytes.

Output For each map, print the square where the treasure is buried, or *WALK THE PLANK* if the map is fake.

Sample Input

```
6
1 ahoy reef map storm skull treasure
9 tide loot island cannon parrot
5 parrot anchor
1 ahoy matey
```

```
1 ahoy loot crew
5 cannon
```

Sample Output

```
7
4
7
WALK THE PLANK
WALK THE PLANK
WALK THE PLANK
```